

MindSpore Transformers 大模型系列课程

MindSpore Transformers & vLLM部署与评测

主讲人：Erpim

专题一：套件介绍

《MindSpore Transformers训推Mcore架构介绍》

《MindSpore Transformers环境安装与镜像制作》

专题二：LLM训练

《MindSpore Transformers LLM预训练与微调介绍与实践》

《MindSpore Transformers LLM数据预处理介绍与实践》

《MindSpore Transformers Safetensors权重详解》

《MindSpore Transformers断点续训介绍与实践》

《MindSpore Transformers训练在线监控介绍与实践》

《MindSpore Transformers训练高可用特性介绍》

专题三：LLM推理

《MindSpore Transformers推理介绍与实践》

《MindSpore Transformers & vLLM部署与评测》

专题四：高阶开发

《MindSpore Transformers LLM推理迁移与实践》

《MindSpore Transformers & vLLM推理精度比对》

《MindSpore Transformers LLM训练迁移与实践》

《MindSpore Transformers & Megatron-LM训练精度比对》

《MindSpore Transformers训练性能调优》

《MindSpore Transformers推理性能调优》

《MindSpore Transformers DeepSeek-R1蒸馏实践》

目录

1. MindSpore Transformers x vLLM-MindSpore插件

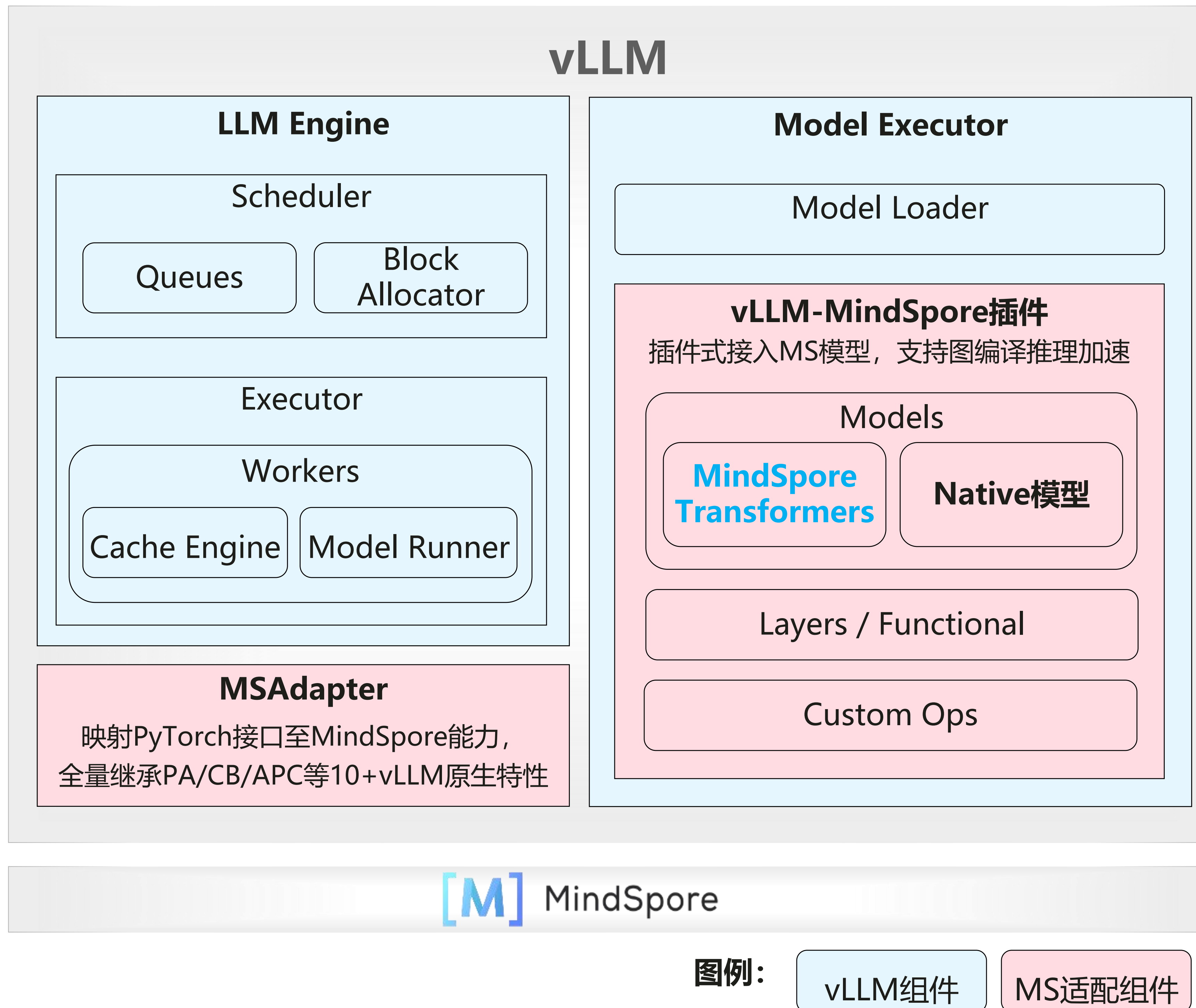
使能服务化推理流程架构

2. 基于vLLM-MindSpore插件实现服务化安装部署流程

3. vLLM-MindSpore插件关键特性原理及演示

4. 评测工具使用示例

MindSpore Transformers x vLLM-MindSpore插件 使能服务化推理流程架构



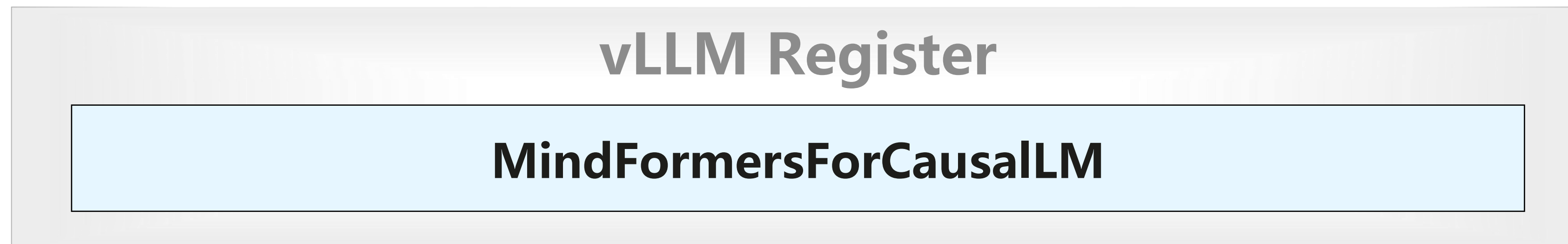
vLLM-MindSpore插件是一个由MindSpore社区孵化的vLLM后端插件。其将基于MindSpore构建的大模型推理能力接入vLLM，从而有机整合MindSpore和vLLM的技术优势，提供**全栈开源、高性能、易用**的大模型推理解决方案。

服务化组件: 通过将LLM Engine、Scheduler等服务化组件中的PyTorch API调用映射至MindSpore能力调用，继承支持包括Continuous Batching、PagedAttention在内的服务化功能。

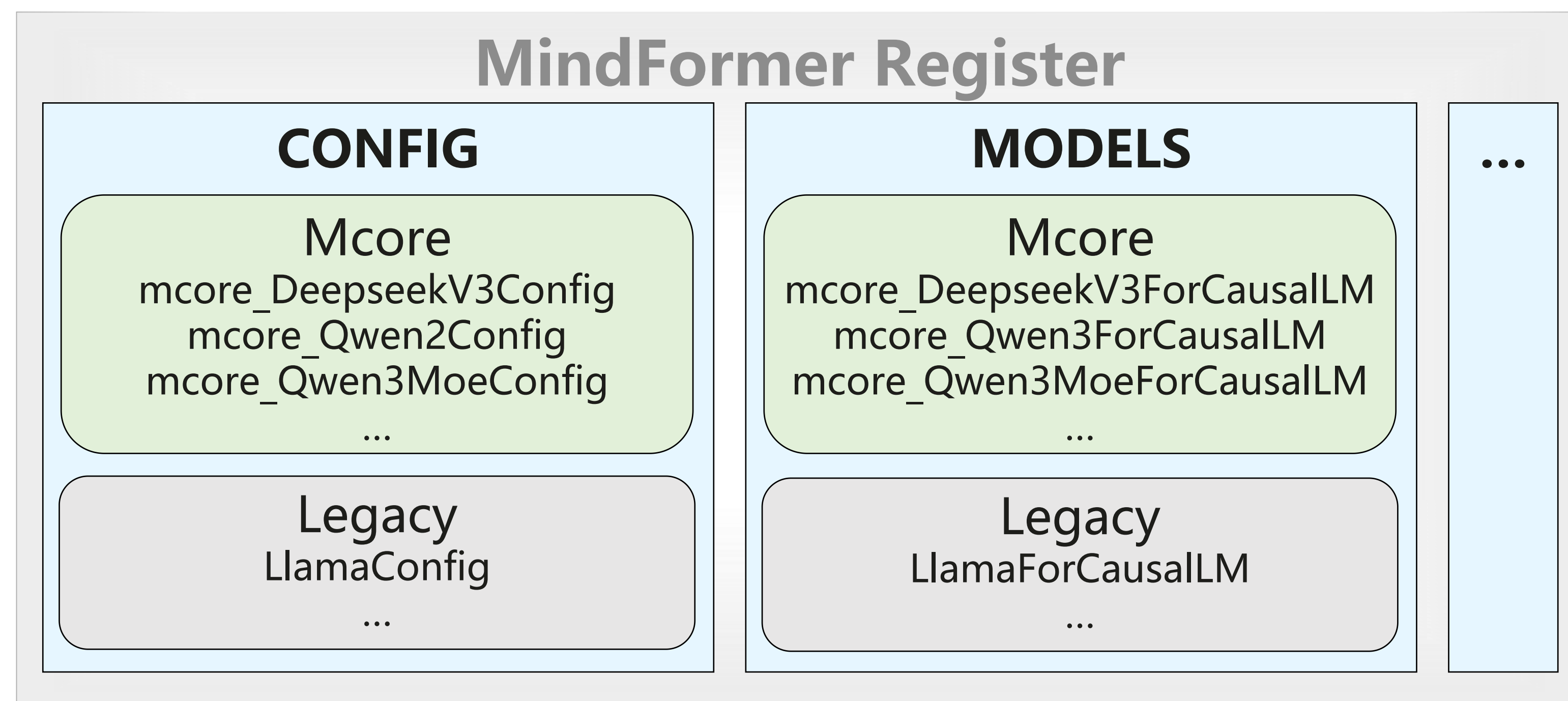
大模型组件: 通过注册或替换模型、网络层、自定义算子等组件，将**MindSpore Transformers**、MindSpore One等MindSpore大模型套件和自定义Native大模型接入vLLM。



服务化侧注册统一模型



套件库侧分发配置和模型



ModelExecutor

Config (Hugging Face & vLLM)

- 提供服务化能力，执行推理模型前向；
- 提供注册接口，其中MindSpore Transformers 注册统一Mcore流程接口MindFormersForCausalLM
- 处理HF Config信息和vLLM CLI启动指令，转换为MF可处理形式；

AutoModel

Network

HF config.json

- model_type
- architectures

- 通过AutoModel将MF模型注册至vLLM，新增模型无需关注服务侧；
- 动静态图统一（MSJit），默认使能MindSpore图编译、运行时和融合算子优化，提供开箱即用的推理性能；

安装方式一：源码安装方式

Step1: 下载并安装CANN包（从MindSpore官网获取）



```
bash Ascend-cann-toolkit_*.run --install -q
bash Ascend-cann-kernels_*.run --install -q
bash Ascend-cann-nnrt_*.run --install -q
```

Step2: 设置CANN相关环境变量



```
source /usr/local/Ascend/ascend-toolkit/set_env.sh
```

Step3: 下载vLLM-MindSpore插件



```
git clone https://gitee.com/mindspore/vllm-
mindspore.git
cd vllm-mindspore
```

Step4: 安装vLLM-MindSpore插件及依赖包



```
bash install_depend_pkgs.sh
pip install .
```

安装方式二：Docker安装方式

Step1: 下载插件源码，启动docker镜像构建



```
git clone https://gitee.com/mindspore/vllm-mindspore.git
cd vllm-mindspore
bash build_image.sh
```

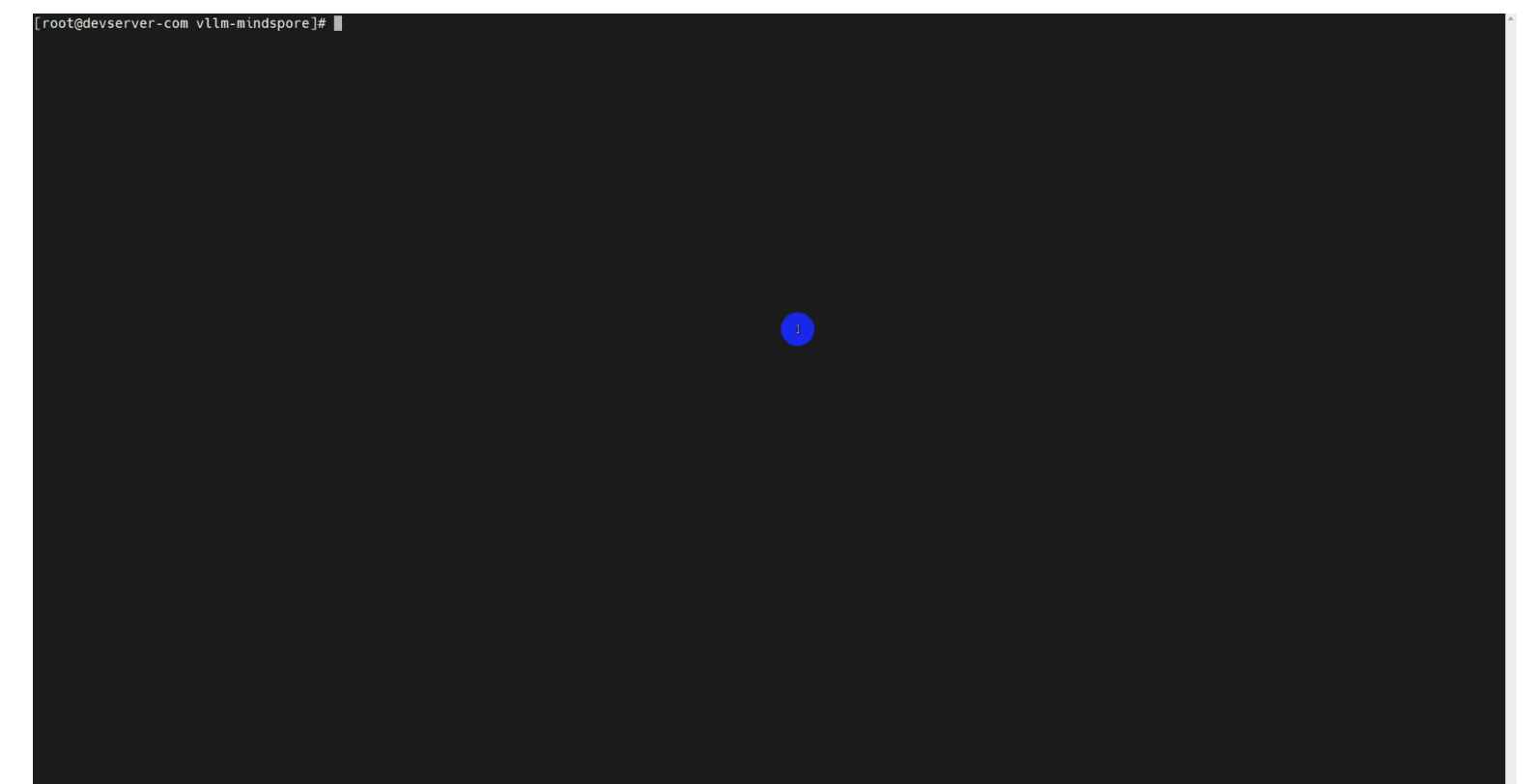
Step2: 创建docker容器并进入容器



```
docker run -itd --name=${DOCKER_NAME} --ipc=host --network=host \
--privileged=true \
--device=/dev/davinci0 --device=/dev/davinci1 \
--device=/dev/davinci2 --device=/dev/davinci3 \
--device=/dev/davinci4 --device=/dev/davinci5 \
--device=/dev/davinci6 --device=/dev/davinci7 \
--device=/dev/davinci_manager --device=/dev/devmm_svm \
--device=/dev/hisi_hdc \
-v /usr/local/sbin:/usr/local/sbin/ \
-v /var/log/npu/slog:/var/log/npu/slog \
-v /var/log/npu/profiling:/var/log/npu/profiling \
-v /var/log/npu/dump:/var/log/npu/dump \
-v /var/log/npu:/usr/slog \
-v /etc/hccn.conf:/etc/hccn.conf \
-v /usr/local/bin/npu-smi:/usr/local/bin/npu-smi \
-v /usr/local/dcmi:/usr/local/dcmi \
-v /usr/local/Ascend/driver:/usr/local/Ascend/driver \
-v /etc/ascend_install.info:/etc/ascend_install.info \
-v /etc/vnpu.cfg:/etc/vnpu.cfg \
-v /home/ckpt:/home/ckpt \
--shm-size="250g" \
${IMAGE_NAME} \
bash
```

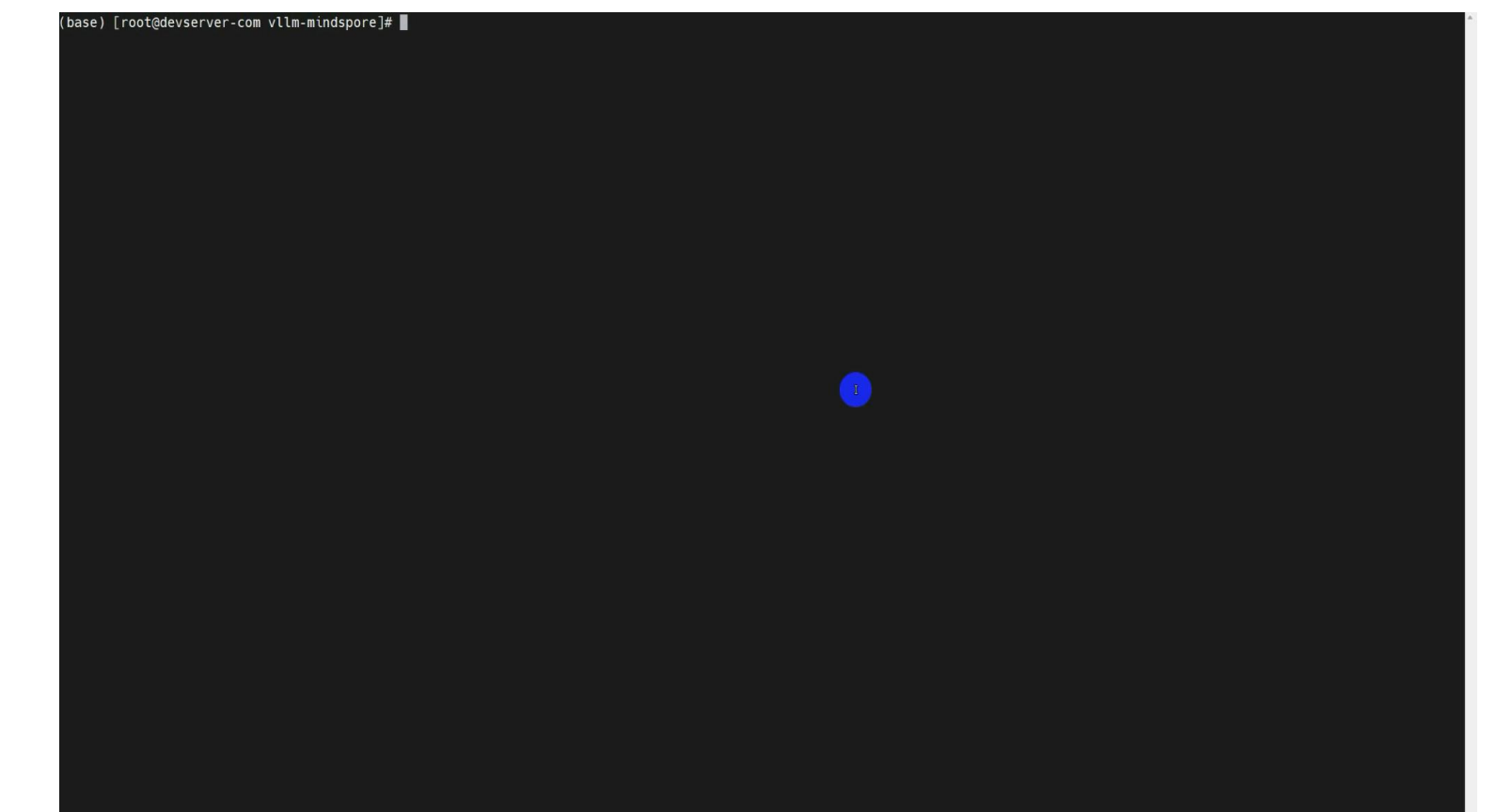
```
docker exec -it $DOCKER_NAME bash
```

源码安装方式



拉取vLLM MindSpore源码，源码编译并安装

docker安装方式



拉取vLLM MindSpore源码，一键启动docker构建

相关启动参数及调用命令

服务化参数	功能描述
--trust-remote-code	信任来自Huggingface的远程代码，默认值为False
--max-num-seqs {max_num_seqs}	每次迭代的最大序列数，未配置时取值256或者1024(VLLM_USE_V1打开时)
--max-model-len {max_model_len}	模型上下文长度，如果未指定，将自动从模型配置中读取
--max-num-batched-tokens {max_num_batched_tokens}	每次迭代的最大批处理token数，不配置时系统会自动计算
--block-size {block_size}	连续缓存块的令牌数量大小，默认值16
--gpu-memory-utilization {gpu_memory_utilization}	显存利用率，默认值为0.9

启动服务

```
●●●

vllm-mindspore serve ${MODEL_PATH}
--trust-remote-code
--max-num-seqs 32
--max-model-len 4096
--block-size 128
--gpu-memory-utilization 0.9
```

发送请求

```
●●●

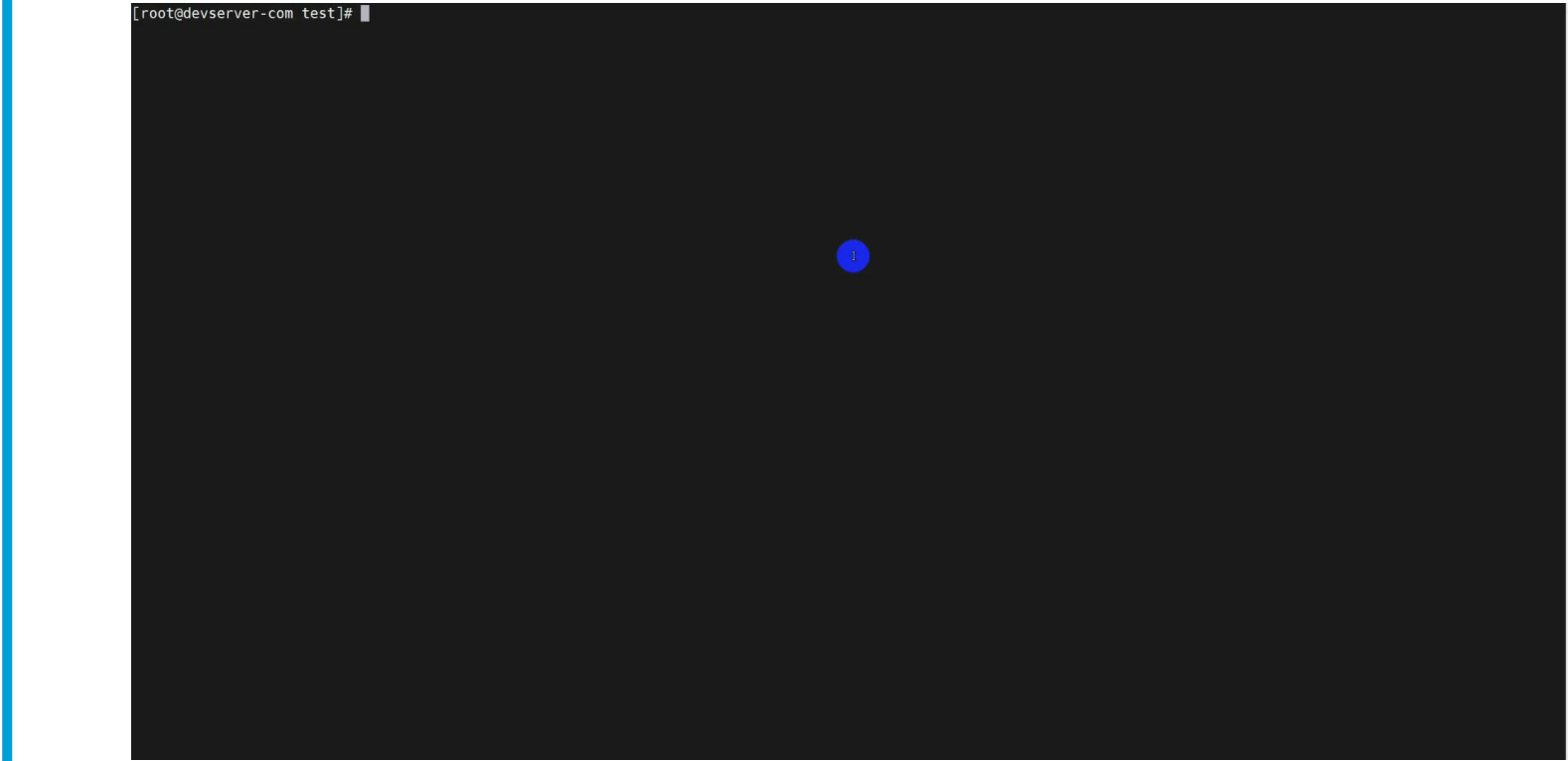
curl -X POST http://localhost:8000/v1/completions
-H "Content-Type: application/json"
-d '{
  "model": "/home/ckpt/Qwen3-8B",
  "temperature": 0,
  "max_tokens": 20,
  "prompt": "Qwen3 is"
}'
```

请求结果

```
●●●

{
  "id": "cmpl-c201bd82c42546acba6d0279a6358827",
  "object": "text_completion",
  "created": 1761703974,
  "model": "/home/ckpt/Qwen3-8B/",
  "choices": [
    {
      "index": 0,
      "text": " a large-scale language model that can be used for various tasks, such as text generation, answering questions",
      "logprobs": null,
      "finish_reason": "length",
      "stop_reason": null,
      "prompt_logprobs": null
    }
  ],
  "usage": {
    "prompt_tokens": 4,
    "total_tokens": 24,
    "completion_tokens": 20,
    "prompt_tokens_details": null
  },
  "kv_transfer_params": null
}
```

模型权重下载



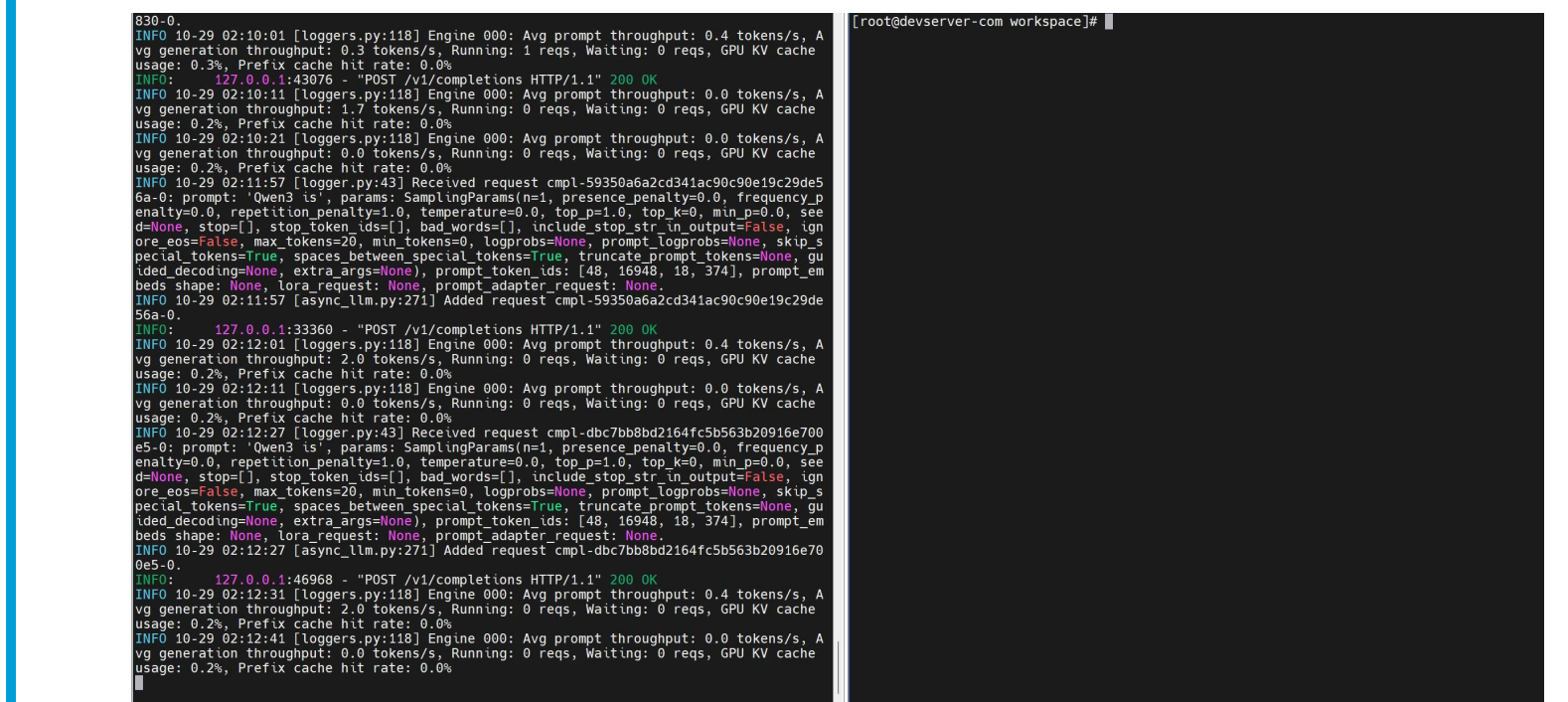
可直接使用Hugging Face权重

启动推理服务



以Qwen3为例，使用vLLM MindSpore启动推理服务

测试推理服务

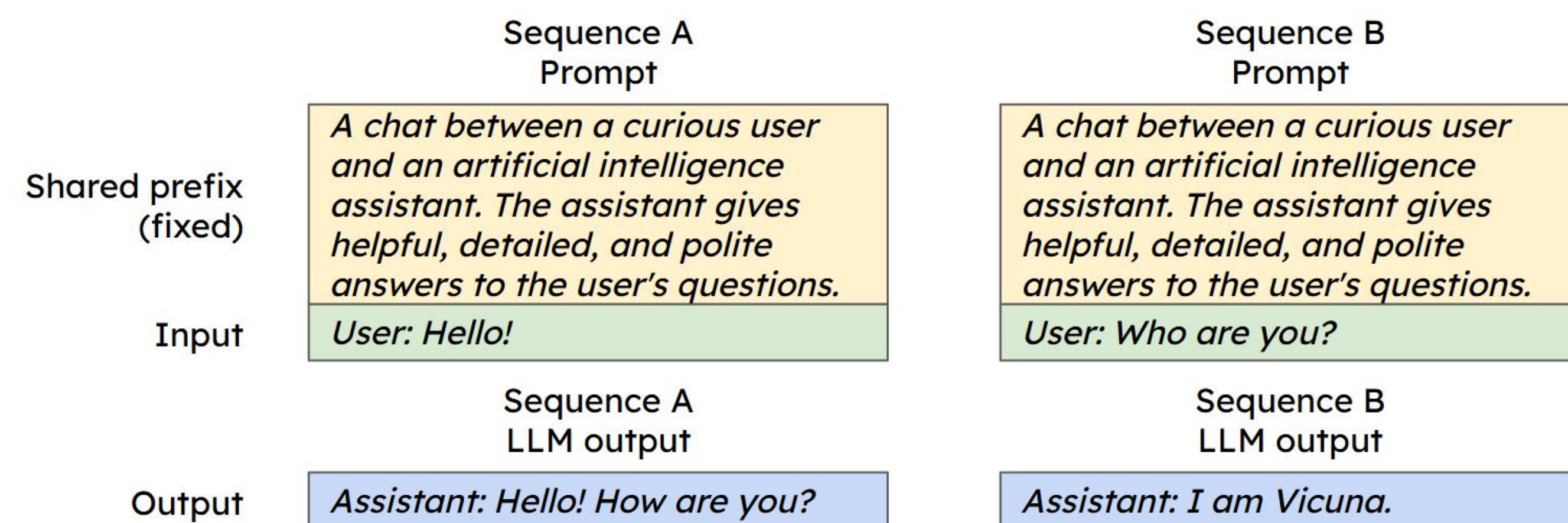


推理服务启动成功后，发送请求测试服务

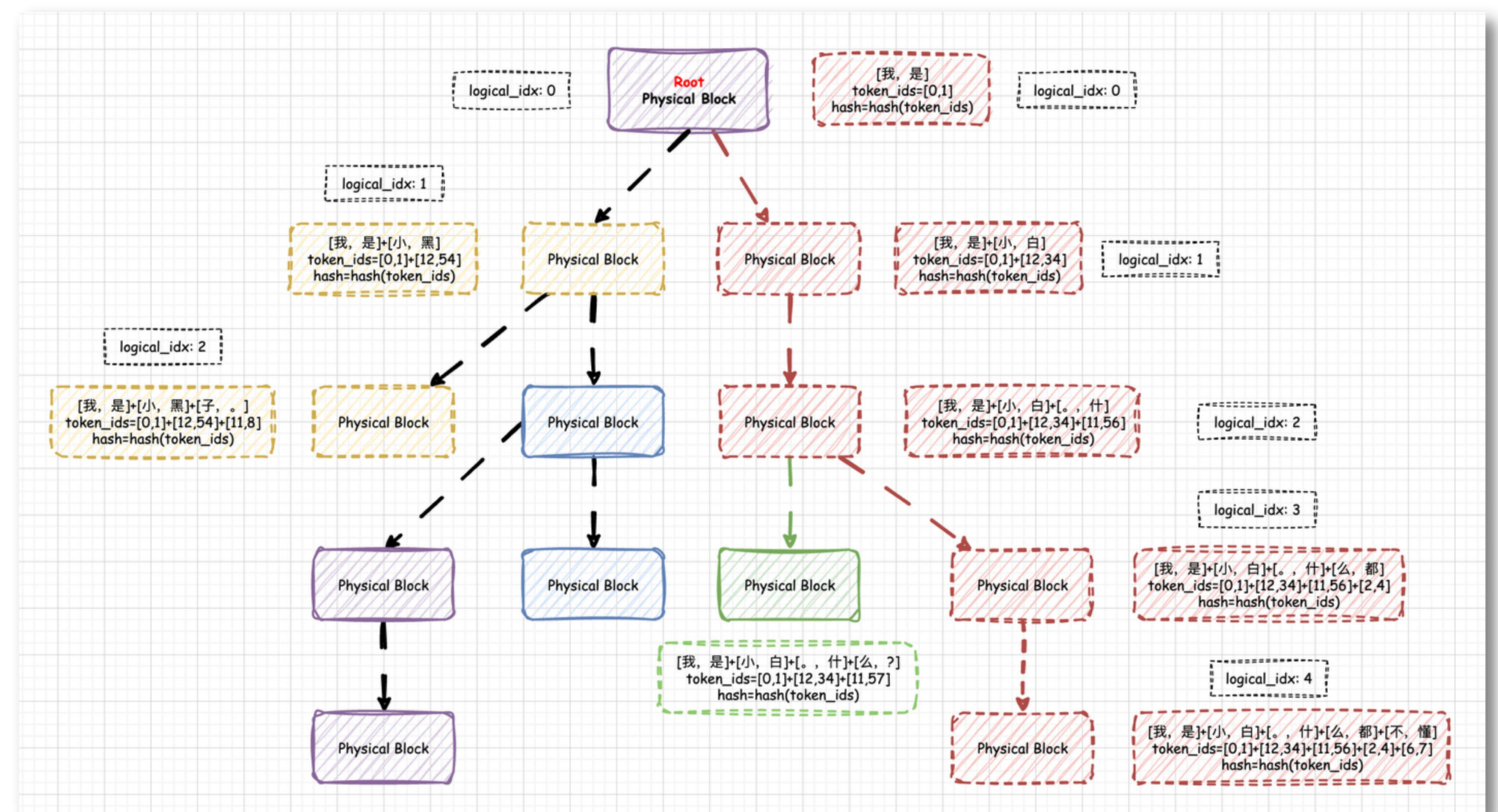
类别	特性名称	核心目标	关键价值
推理效率优化	Prefix Caching	降延迟	复用重复计算，提升短序列响应速度
	Chunked Prefill	提长序列效率	缓解显存压力，支持超长文本推理
模型适配与定制	Multi-LoRA	多任务适配	轻量化定制，降低多任务部署成本
工具交互能力	Function Call	扩能力边界	对接外部工具，实现复杂任务执行
资源部署优化	混合并行	大模型部署	突破单设备限制，支持超大规模模型
	模型量化	低成本部署	适配低算力设备，降低硬件投入

推理效率优化——Prefix Caching

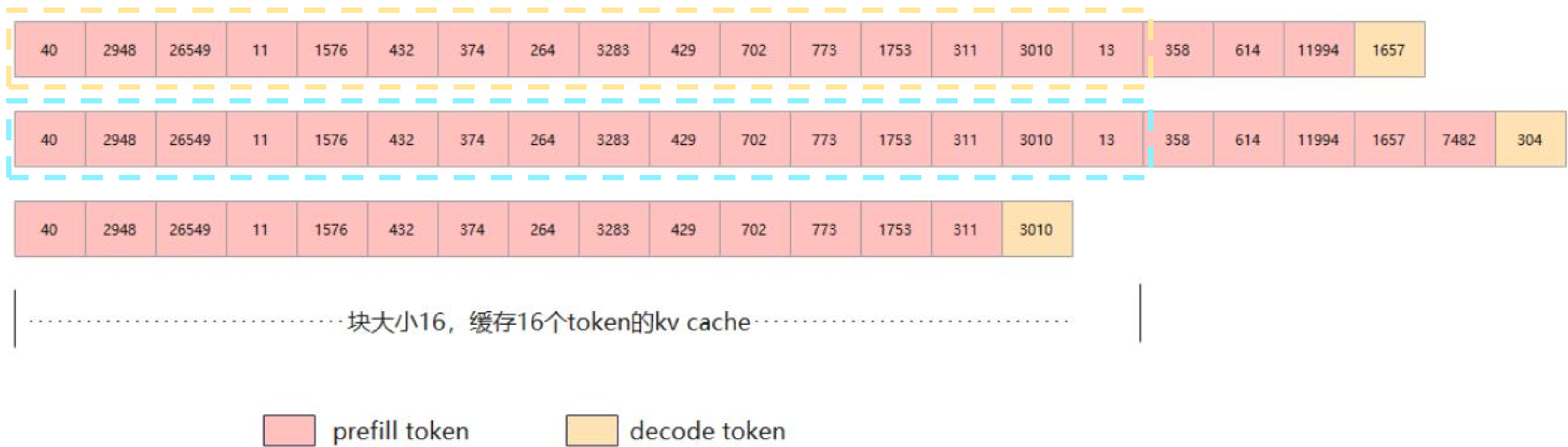
Prefix Caching（前缀缓存）是一种专为大语言模型（LLM）推理优化设计的技术，其核心思想是通过缓存历史请求中的公共前缀部分，避免重复计算，从而显著提升推理效率，尤其TTFT。多个prompt在**相同位置（position）**出现了**同样的序列（tokens）**，复用KV Cache的计算和存储。



- **多轮对话场景：**每一轮对话需要依赖所有历史轮次对话的上下文
- **Shared prefix：**共享前置信息，eg：system message



Prefix Caching 服务化部署示例



请求1：缓存前16个token

请求2：命中前16个token

请求3：不满块未命中

相关启动参数及调用命令

服务化参数	功能描述
--block-size {block_size}	连续缓存块的令牌数量大小。
--enable-prefix-caching	启用前缀缓存，针对V0流程生效。V1流程默认使能。
--no-enable-prefix-caching	关闭前缀缓存，针对V1流程生效。

```
python3 -m vllm_mindspore.entrypoints vllm.entrypoints.openai.api_server
--model "/home/ckpt/Qwen3-8B"
--tensor_parallel_size=1
--block-size=16
--enable-prefix-caching
```

服务化示例

前缀缓存效果演示



多次发送具有相同前缀的请求，缓存命中率升高

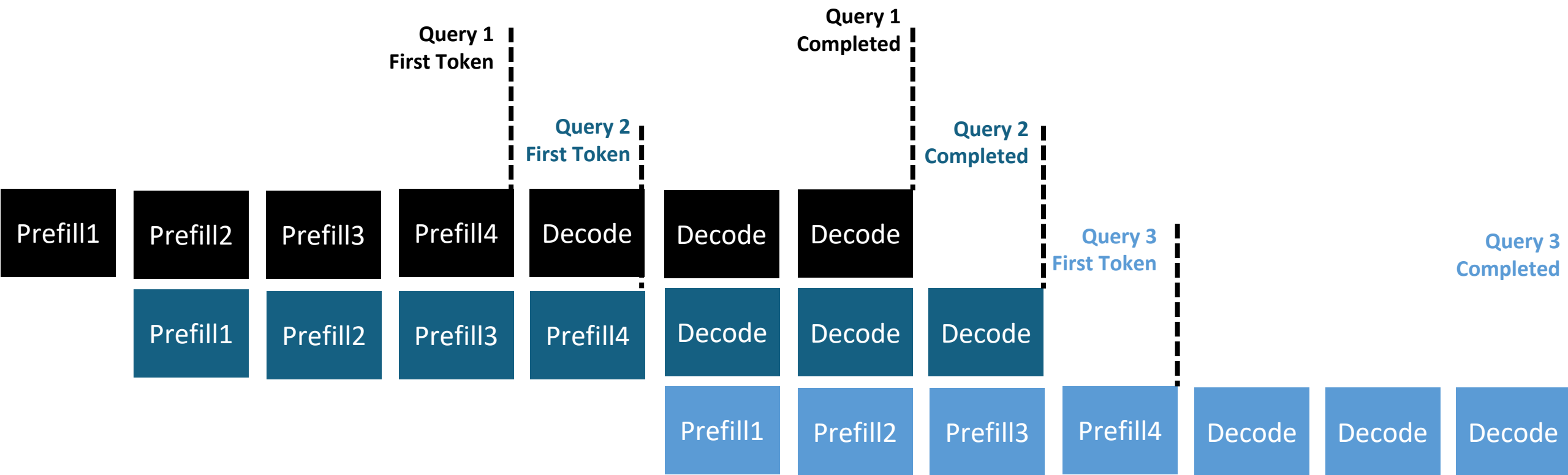
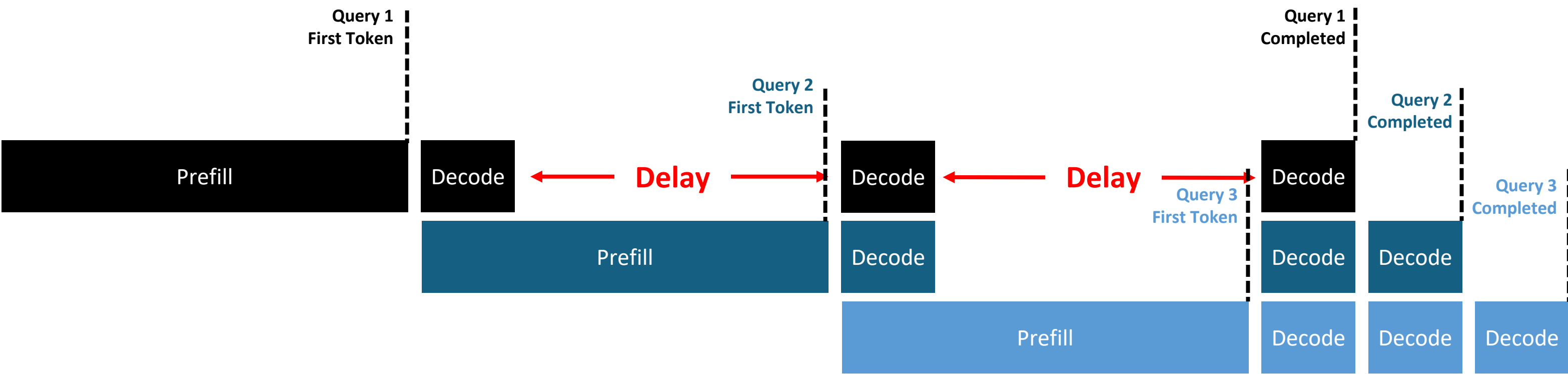
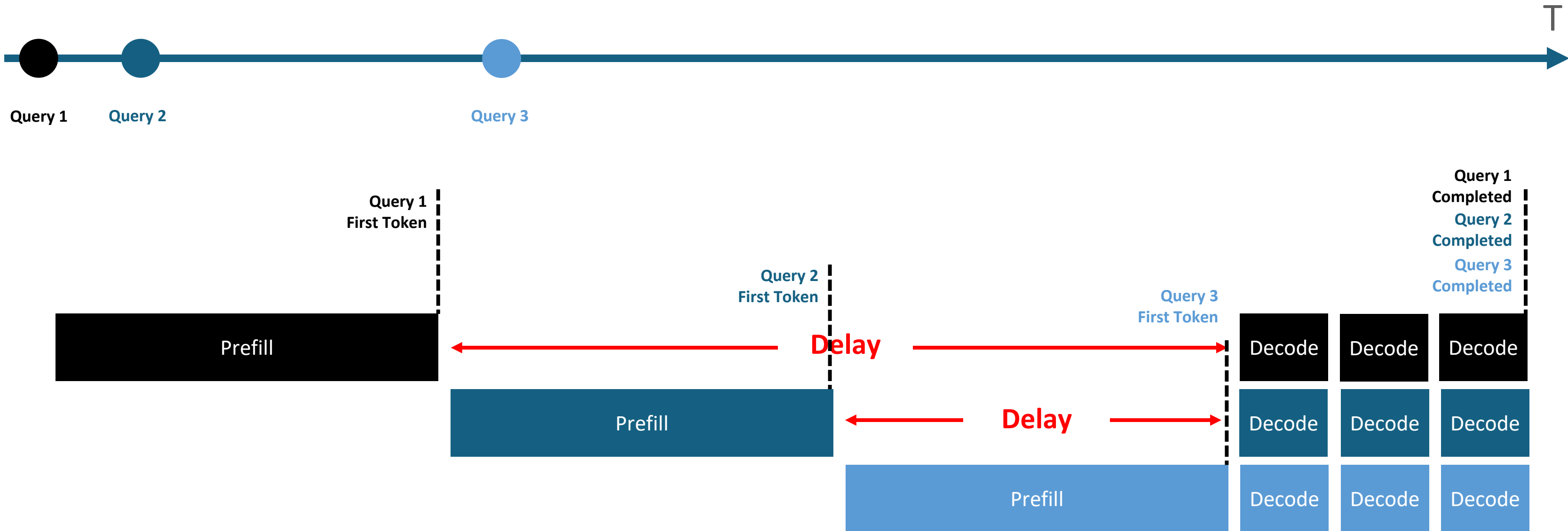
Chunked Prefill（分块预填充）是大语言模型（LLM）推理优化的核心技术之一，其核心思想是将长输入序列的预填充（Prefill）阶段**拆分为多个小块**，与解码（Decode）任务**交错执行**，从而提升Device利用率、降低延迟并优化显存管理。

相关启动参数及调用命令

服务化参数	功能描述
--max-num-batched-tokens {size}	单次迭代中要处理的最大令牌数。
--enable-chunked-prefill	是否启用Chunked Prefill针对V0流程生效。V1流程默认使能。

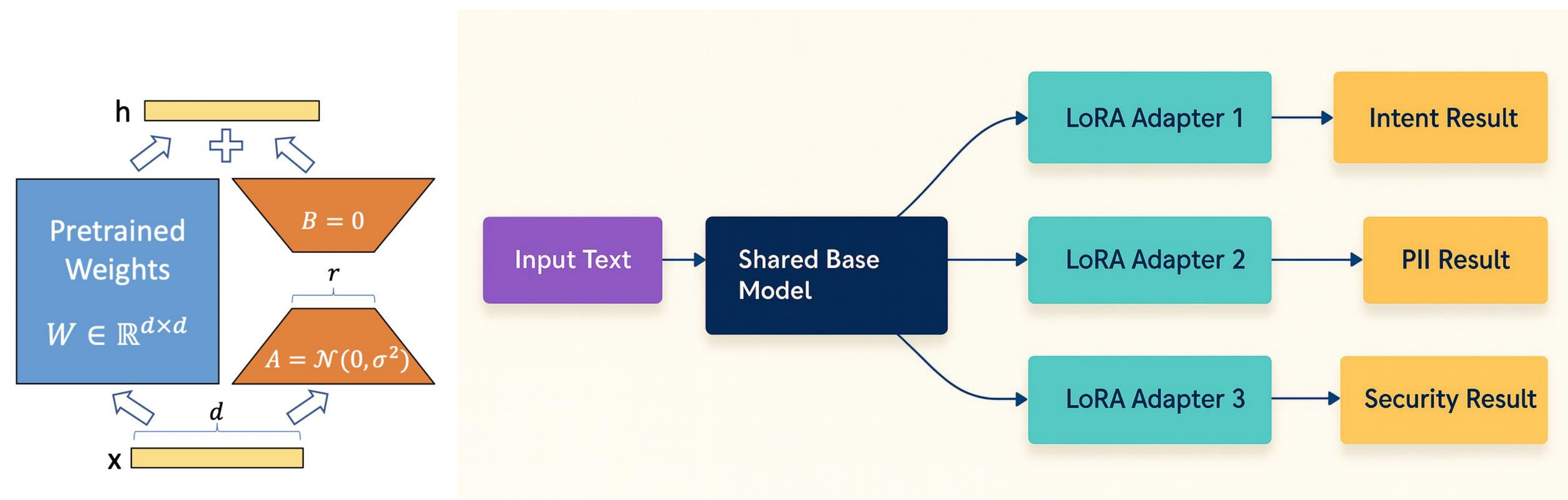
```
export VLLM_USE_V1=0

vllm-mindspore serve Qwen3-8B
--trust-remote-code
--max-num-seqs 32
--max-model-len 16384
--block-size 128
--gpu-memory-utilization 0.9
--max-num-batched-tokens 2048
--enable-chunked-prefill
```



Multi-LoRA 特性在推理场景中通过动态管理多个低秩适配器

(LoRA)，显著提升了大语言模型（LLM）的灵活性、资源效率和多任务处理能力。



传统方案为每个微调任务单独部署模型，显存占用随任务数量线性增长。Multi-LoRA 通过**共享基础模型权重**，仅加载多个轻量化 LoRA 适配器（通常每个仅占基础模型的 0.1-1% 显存），实现「**一基多调**」的高效显存利用。

初始化阶段

加载基础模型。

预加载多个领域特定的LoRA适配器（如医疗、法律等）。

接收请求

服务器同时接收多个用户请求，每个请求指定使用的LoRA适配器。

示例：

请求1：用户提问“血液有哪些生理功能？”，指定使用医疗LoRA适配器。

请求2：用户提问“违章停车与违法停车是否有区别？”，指定使用法律LoRA适配器。

动态适配

vLLM根据每个请求指定的LoRA适配器，动态切换到对应的领域权重。

每个请求独立处理，互不干扰，确保领域专业性。

生成响应

使用切换后的模型（基础模型+特定LoRA权重）生成对应领域的专业回答。

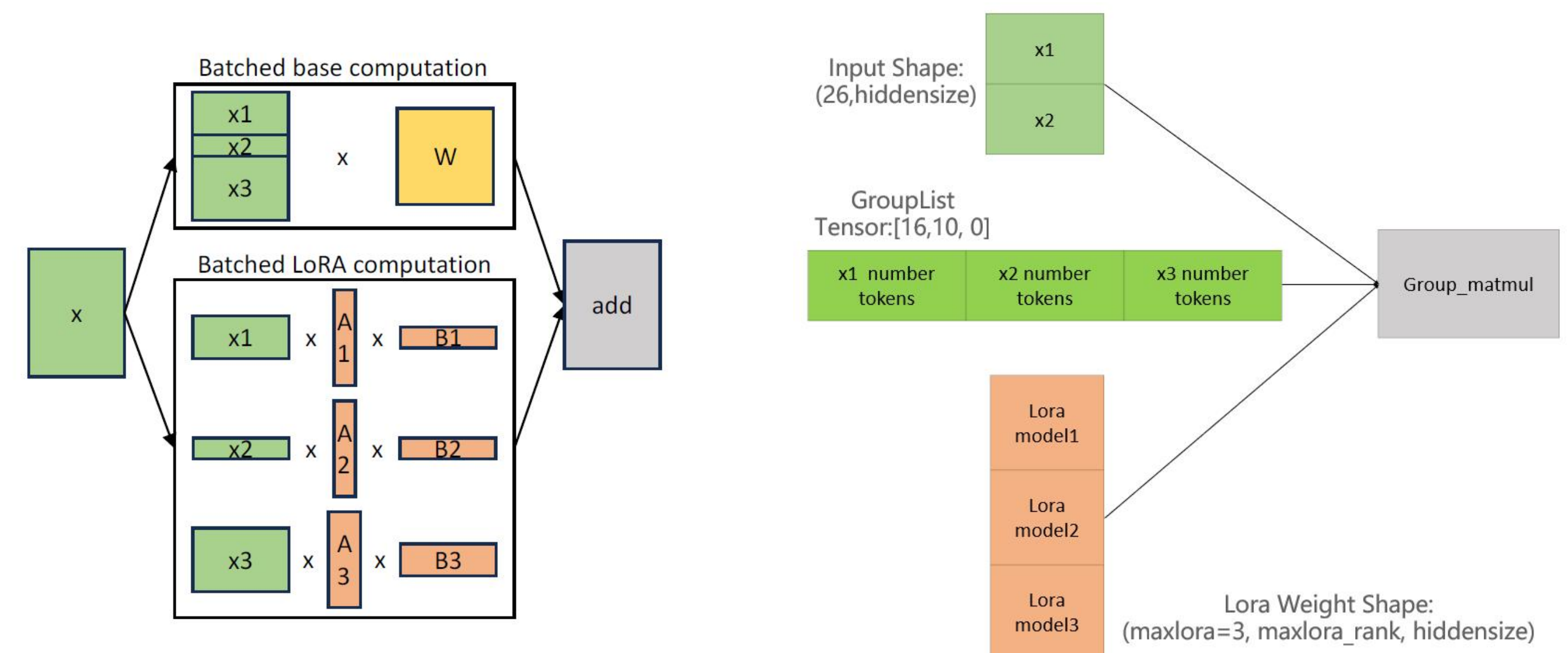
示例结果：

医疗问题：生成包含血液运输、调节、防御等生理功能的专业回答。

法律问题：生成从法律定义、处罚依据、后果等方面区分的专业回答。

输出结果

将适配后的专业回答返回给对应请求的用户。



相关启动参数及调用命令

服务化参数	功能描述
--enable-lora	启用 LoRA 适配器处理
--max-loras {size}	单个批次中 LoRA 的最大数量
--max-lora-rank {rank_size}	最大 LoRA 等级
--lora-modules {lora_config}	LoRA 模块配置，可以是 'name=path' 格式或 JSON 格式或 JSON 列表格式。 示例（旧格式）：'name=path' 示例（新格式）：{'name': 'name', 'path': 'lora_path', 'base_model_name': 'id'}



```
vllm-mindspore serve /home/ckpt/qwen2.5-7b-hf
--tensor_parallel_size=1
--gpu-memory-utilization=0.9
--max_model_len=800
--enable-lora
--lora-modules lora1='/home/ckpt/qwen2.5-7b-lora-law'
lora2='/home/ckpt/qwen2.5-7b-lora-medical'
--max_loras=3
--max_lora_rank=64
```

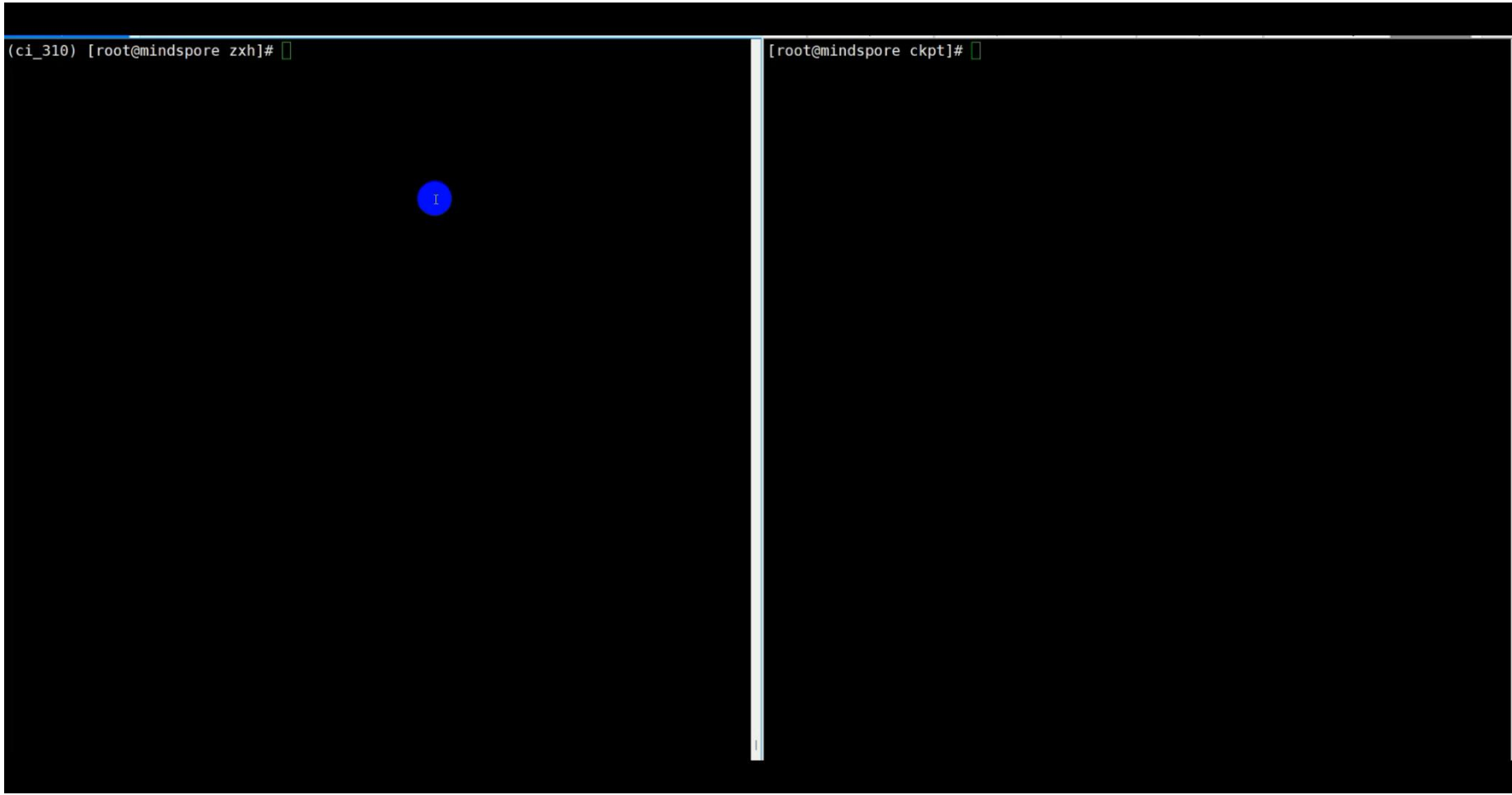
服务化示例



```
# 请求1:
curl http://localhost:8000/v1/completions -H "Content-Type: application/json" -d
'{"model": "/home/ckpt/qwen2.5-7b-hf", "prompt": "违章停车与违法停车是否有区别?",
"model": "lora1", "max_tokens": 20, "temperature": 0, "top_p": 1.0, "top_k": -1,
"repetition_penalty": 1.0}'

请求2:
curl http://localhost:8000/v1/completions -H "Content-Type: application/json" -d
'{"model": "/home/ckpt/qwen2.5-7b-hf", "prompt": "血液有哪些成分?", "model": "lora2",
"max_tokens": 20, "temperature": 0, "top_p": 1.0, "top_k": -1,
"repetition_penalty": 1.0}'
```

Multi-LoRA示例



查询两个专业领域问题

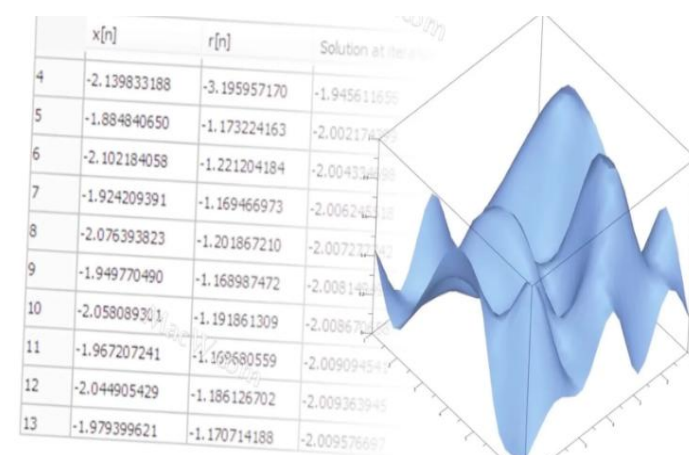
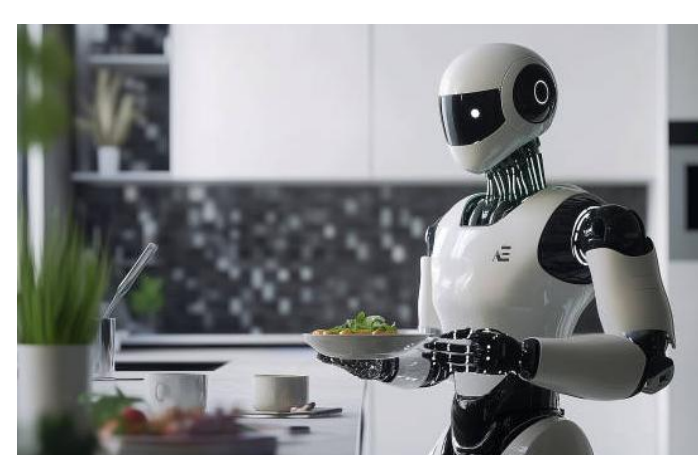
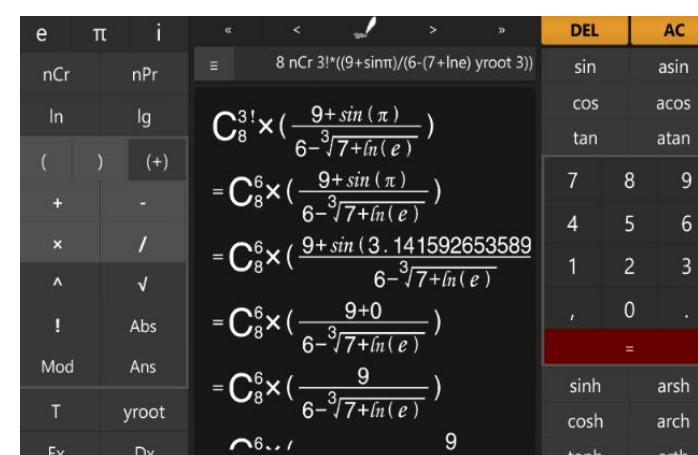
功能概述

Function call 功能最早由 GPT-4 引入，该功能允许模型通过调用特定的外部工具 (tools) 来执行某些复杂任务，这些 tools 通常是开发者定义的函数。具有 function call 能力的模型可以根据提问智能地选择 tools 并输出一个包含调用这些 tools 所需参数的 JSON 对象。

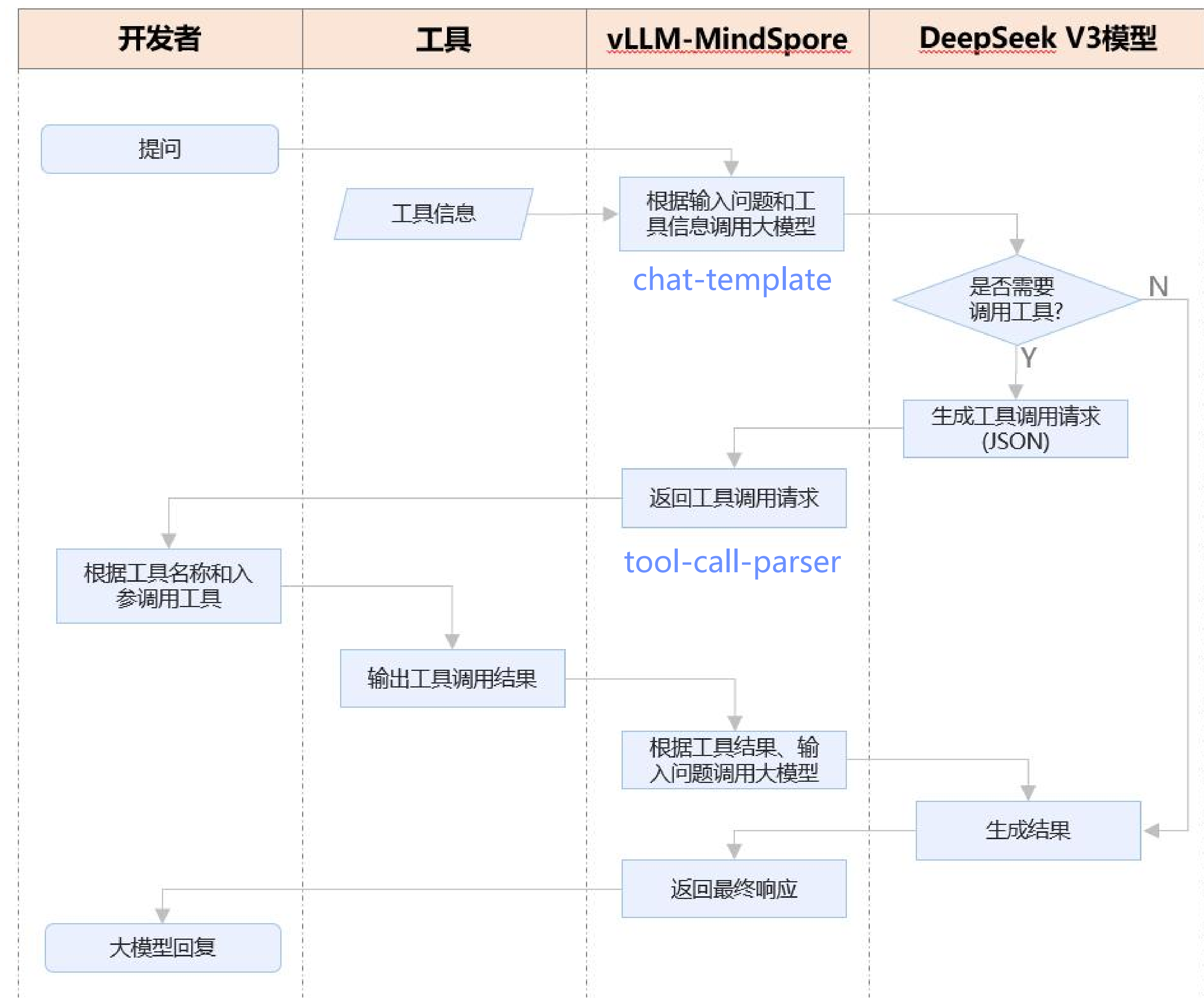
用户需求：“查一下今天深圳的实时气温”

- ↓
1. 模型推理：
 - 识别需求：需获取“深圳+今日+实时气温”数据
 - 生成工具调用指令：调用「天气API」，参数为“城市=深圳，时间=今日，类型=实时气温”
- ↓
2. 工具响应：天气API返回“深圳今日实时气温28°C，晴”
- ↓
3. 模型推理：将API返回内容转化为自然语言（无二次推理）
- ↓
- 最终回复：“今天深圳的实时气温是28°C，天气晴朗~”

应用场景



实时 / 动态信息获取 复杂数据处理与计算 物理世界控制与交互 专业领域工具调用



相关启动参数及调用命令

服务化参数	功能描述
--enable-auto-tool-choice	用于启用自动工具选择功能
--chat-template {template_file}	指定聊天交互的模板格式，如/path/tool_chat_template_deepseekv3.jinja
--tool-call-parser {parser_name}	用于控制function call时如何解析和处理工具调用的相关信息，当前仅支持配置"deepseek_v3"

```
vllm-mindspore serve /hf_quant_ckpt/V3-0324-A8W8
--trust_remote_code
--tensor_parallel_size=16
--max-num-seqs=192
--max_model_len=65536
--max-num-batched-tokens=4096
--block-size=128
--gpu-memory-utilization=0.9
--quantization golden-stick
--enable-chunked-prefill
--enable-auto-tool-choice
--tool-call-parser deepseek_v3
--chat-template tool_chat_template.jinja
```

服务化示例

用户请求：查一下深圳的实时天气

生成工具输入

```
INFO 10-29 10:37:37 [logger.py:43] Received request chatcmpl-63d3ac44923f46038ce6d0dd10b10213: prompt: '\n<|begin_of_sentence|>\n\n\n你可以调用工具函数
。当你需要调用工具时，你必须严格遵守下面的格式输出：<|tool_calls_begin|><|tool_call_begin|>function<|tool_sep|>FUNCTION_NAME\n```\njson\n{"param1": "
value1", "param2": "value2"}\n```\n<|tool_call_end|><|tool_calls_end|>\n\n\n不遵守上面的格式就不能成功调用工具，是错误答案。
\n错误答案举例1: function<|t
ool_sep|>FUNCTION_NAME\n```\njson\n{"param1": "value1", "param2": "value2"}\n```\n<|tool_call_end|><|tool_calls_end|>\n\n\n错误1原因：没有使用<|tool_calls
_begin|>、<|tool_call_begin|>，不符合格式。
\n错误答案举例2: <|tool_calls_begin|><|tool_call_begin|>function<|tool_sep|>FUNCTION_NAME\n```\njson\n{"
param1": "value1", "param2": "value2"}\n```\n<|tool_call_end|>\n\n\n错误2原因：没有使用<|tool_calls_end|>，不符合格式。
\n错误答案举例3: <|tool_calls_begin
|><|tool_call_begin|>function<|tool_sep|>FUNCTION_NAME\n```\njson\n{"param1": "value1", "param2": "value2"}\n```\n<|tool_call_end|><|tool_calls_begin
|>\n\n\n错误3原因：最后一个<|tool_calls_begin|>应为<|tool_calls_end|>，不符合格式。
## Tools\n\n\n## Function\n\n\nYou have the following functions availabl
e:\n\n\n```\njson\n{"type": "function", "function": {"name": "get_current_weather", "description": "Get the current weather in a given location", "paramete
rs": {"type": "object", "properties": {"city": {"type": "string", "description": "The city to find the weather for, e.g. \\'San Francisco\'}}, "unit": {"t
ype": "string", "description": "The unit to fetch the temperature in", "enum": ["celsius", "fahrenheit"]}}, "required": ["city", "unit"]}}}\n```\n\n<|User
|>查一下深圳的实时天气<|Assistant|>
, params: SamplingParams(n=1, presence_penalty=0.0, frequency_penalty=0.0, repetition_penalty=1.0, temperature=1.0
, top_p=1.0, top_k=0, min_p=0.0, seed=None, stop=[], stop_token_ids=[], bad_words=[], include_stop_str_in_output=False, ignore_eos=False, max_tokens=6519
1, min_tokens=0, logprobs=None, prompt_logprobs=None, skip_special_tokens=True, truncate_prompt_tokens=None, guided_d
ecoding=None, extra_args=None), prompt_token_ids: None, prompt_embeds shape: None, lora_request: None, prompt_adapter_request: None.
INFO 10-29 10:37:37 [async_llm.py:271] Added request chatcmpl-63d3ac44923f46038ce6d0dd10b10213.
-----model output:
<|tool_calls_begin|><|tool_call_begin|>function<|tool_sep|>get_current_weather
```\n
json
{"city": "深圳", "unit": "celsius"}
```\n<|tool_call_end|><|tool_calls_end|>
```

根据输入生成工具的输入

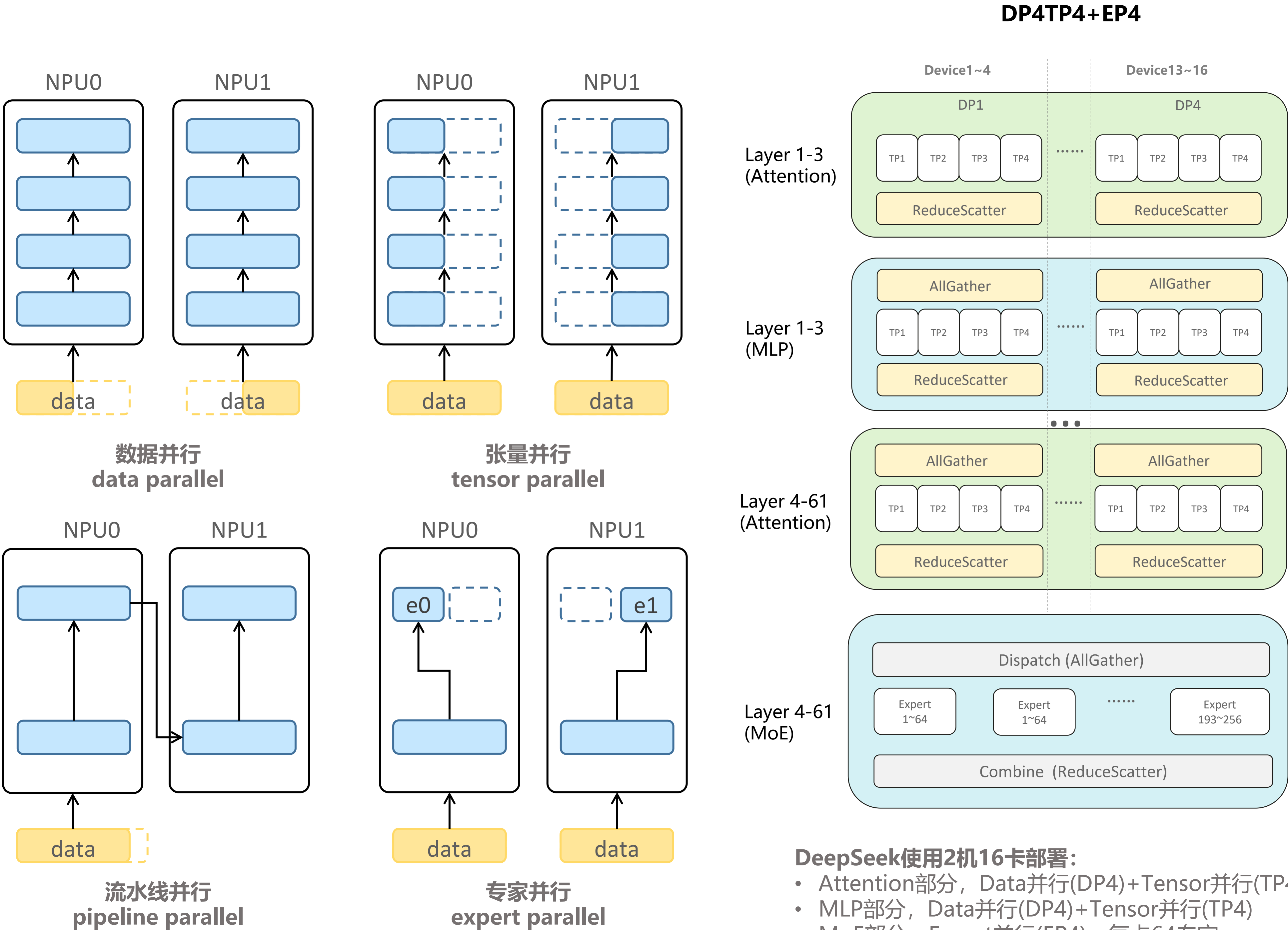
反馈最终结果

```
INFO: 127.0.0.1:49742 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO 10-29 10:37:38 [logger.py:43] Received request chatcmpl-c510f202313e4d5da0cc46a881e6efa7: prompt: '\n<|begin_of_sentence|>\n\n\n你可以调用工具函数
。当你需要调用工具时，你必须严格遵守下面的格式输出：<|tool_calls_begin|><|tool_call_begin|>function<|tool_sep|>FUNCTION_NAME\n```\njson\n{"param1": "
value1", "param2": "value2"}\n```\n<|tool_call_end|><|tool_calls_end|>\n\n\n不遵守上面的格式就不能成功调用工具，是错误答案。
\n错误答案举例1: function<|t
ool_sep|>FUNCTION_NAME\n```\njson\n{"param1": "value1", "param2": "value2"}\n```\n<|tool_call_end|><|tool_calls_end|>\n\n\n错误1原因：没有使用<|tool_calls
_begin|>、<|tool_call_begin|>，不符合格式。
\n错误答案举例2: <|tool_calls_begin|><|tool_call_begin|>function<|tool_sep|>FUNCTION_NAME\n```\njson\n{"
param1": "value1", "param2": "value2"}\n```\n<|tool_call_end|>\n\n\n错误2原因：没有使用<|tool_calls_end|>，不符合格式。
\n错误答案举例3: <|tool_calls_begin
|><|tool_call_begin|>function<|tool_sep|>FUNCTION_NAME\n```\njson\n{"param1": "value1", "param2": "value2"}\n```\n<|tool_call_end|><|tool_calls_begin
|>\n\n\n错误3原因：最后一个<|tool_calls_begin|>应为<|tool_calls_end|>，不符合格式。
## Tools\n\n\n## Function\n\n\nYou have the following functions availabl
e:\n\n\n```\njson\n{"type": "function", "function": {"name": "get_current_weather", "description": "Get the current weather in a given location", "paramete
rs": {"type": "object", "properties": {"city": {"type": "string", "description": "The city to find the weather for, e.g. \\'San Francisco\'}}, "unit": {"t
ype": "string", "description": "The unit to fetch the temperature in", "enum": ["celsius", "fahrenheit"]}}, "required": ["city", "unit"]}}}\n```\n\n<|User
|>查一下深圳的实时天气<|Assistant|>
, params: SamplingParams(n=1, presence_penalty=0.0, frequency_penalty=0.0, repetition_penalty=1.0, temperature=1.0
, top_p=1.0, top_k=0, min_p=0.0, seed=None, stop=[], stop_token_ids=[], bad_words=[], include_stop_str_in_output=False, ignore_eos=False, max_tokens=65142, min_tokens=0, logprobs=None, prompt_logprobs=None, skip_special_tokens=True, spaces_between
_special_tokens=True, truncate_prompt_tokens=None, guided_decoding=None, extra_args=None), prompt_token_ids: None, prompt_embeds shape: None, lora_reques
t: None, prompt_adapter_request: None.
INFO 10-29 10:37:38 [async_llm.py:271] Added request chatcmpl-c510f202313e4d5da0cc46a881e6efa7.
-----model output:
今天深圳的实时天气是晴天，气温为28℃。
```

根据工具的返回值反馈最终结果

为在有限集群资源下高效执行大规模模型推理，服务化支持多种并行策略，并支持多维混合并行部署，保障大模型推理的高效率与可扩展性。其核心思想是根据模型结构、硬件配置和业务需求，灵活分配计算任务，最大化利用 NPU 集群资源。

并行策略	基本介绍	特点	是否支持
数据并行 (DP)	每个设备保存完整的模型副本，输入数据按批次 (batch) 分片，各设备独立处理不同样本	提升吞吐量，适合多请求并发	☑
张量并行 (TP)	将模型中的权重或计算操作（如矩阵乘法）沿特定维度切分到多个设备上协同完成	显著降低每卡模型参数显存，但通信量较大	☑
流水线并行 (PP)	将模型按层数划分为多个阶段，不同设备负责不同层，输入数据像流水线一样依次通过各阶段。	减少每卡层数和显存，但会存在气泡，通信量较小	☑
专家并行 (EP)	不同的“专家网络”被分布到不同设备上，路由机制决定激活哪些专家。	MoE结构专用，能够节省每卡专家参数内存	☑
序列并行 (SP)	以输入序列长度这一维度进行分片，不同设备处理序列的不同部分，并在关键层同步中间结果。	降低长序列引起的激活显存，有效支持长文本生成	开发中



DeepSeek使用2机16卡部署：

- Attention部分，Data并行(DP4)+Tensor并行(TP4)
- MLP部分，Data并行(DP4)+Tensor并行(TP4)
- MoE部分，Expert并行(EP4)，每卡64专家

张量并行相关启动参数

服务化参数	功能描述
--tensor-parallel-size {tp_size}	张量并行数，默认值为1

专家并行相关启动参数

服务化参数	功能描述
--enable-expert-parallel	是否使能专家并行，默认不使能
--additional-config {additional_config}	传入字典，设置expert_parallel字段为专家并行数。如--addition-config '{"expert_parallel": 4}

流水线并行相关启动参数

服务化参数	功能描述
--pipeline-parallel-size {pp_size}	流水线并行数，默认值为1

数据并行相关启动参数

服务化参数	功能描述
--data-parallel-size {dp_size}	数据并行数，默认值为1。
--data-parallel-backend {dp_backend}	设置DP部署方式，可选项为 mp 和 ray。默认行为下，DP 将以 multiprocessing 方式部署。
--data-parallel-size-local {dp_size_local}	当前服务节点中的DP数。当部署方式为mp时需要设置。
--data-parallel-start-rank {dp_start_rank}	当前服务节点中负责的首个DP的偏移量。当部署方式为mp时需要设置。
--data-parallel-address {dp_address}	主节点的通讯IP。当部署方式为mp时需要设置。
--data-parallel-rpc-port {dp_rpc_port}	主节点的通讯端口。当部署方式为mp时需要设置。
--headless	是否为从节点。当部署方式为mp时需要设置。

混合并行多机服务化示例1——ray启动

Step1: 主节点启动ray



```
ray start --head --port=<port-to-ray>
```

```
Local node IP: [REDACTED]
-----
Ray runtime started.
-----
Next steps
To add another node to this Ray cluster, run
  ray start --address='[REDACTED]:6688'

To connect to this Ray cluster:
import ray
ray.init()

To submit a Ray job using the Ray Jobs CLI:
RAY_ADDRESS='http://127.0.0.1:8265' ray job submit --working-dir . -- python my_script.py
```

Step2: 从节点通过ray链接主节点

```
ray start --address=<head_node_ip>:<port>
ray status
```

```
(no pending nodes)
Recent failures:
  (no failures)

Resources
-----
Usage:
  0.0/384.0 CPU
  0.0/16.0 NPU
  0B/2.58TiB memory
  0B/372.53GiB object_store_memory
```

Step3: 主节点启动服务

```
vllm-mindspore serve /data/dsr1-w8a8
--trust-remote-code
--max-num-seqs=256
--max-model-len=16384
--max-num-batched-tokens 2048
--block-size=128
--gpu-memory-utilization=0.93
--tensor-parallel-size 4
--data-parallel-size 4
--data-parallel-size-local 2
--data-parallel-backend=ray
--enable-expert-parallel
--additional-config '{"expert_parallel": 4}'
--quantization ascend
```

混合并行多机服务化示例2——multiprocess启动

Step1: 主节点启动命令

```
vllm-mindsore serve /data/dsr1-w8a8/
--trust_remote_code --host 0.0.0.0 --port 80
--max-num-seqs 256 --max-num-batched-tokens
--max_model_len 16384 --block-size 128 \
--gpu-memory-utilization 0.93 --quantization
--tensor-parallel-size 4 --data-parallel-size
--data-parallel-size-local 2 --data-parallel
--data-parallel-address {head_node_ip} \
--data-parallel-rpc-port 8888 \
--enable-expert-parallel \
--additional-config '{"expert_parallel": 4}'
```

Step2: 从节点启动命令

```
vllm-mindspore serve /data/dsr1-w8a8/
--trust_remote_code --host 0.0.0.0 --port 8000 \
--max-num-seqs 256 --max-num-batched-tokens 2048 \
--max_model_len 16384 --block-size 128 \
--gpu-memory-utilization 0.93 --quantization ascend \
--tensor-parallel-size 4 --data-parallel-size 4 \
--data-parallel-size-local 2 --data-parallel-start-rank 2 \
--data-parallel-address {head_node_ip} \
--data-parallel-rpc-port 8888 \
--enable-expert-parallel --headless \
--additional-config '{"expert_parallel": 4}'
```

DeepSeek部署示例

[illegible]

两机16卡, DP4 TP4 EP4

权重准备

方式一：使用金箍棒将浮点权重转化为量化权重

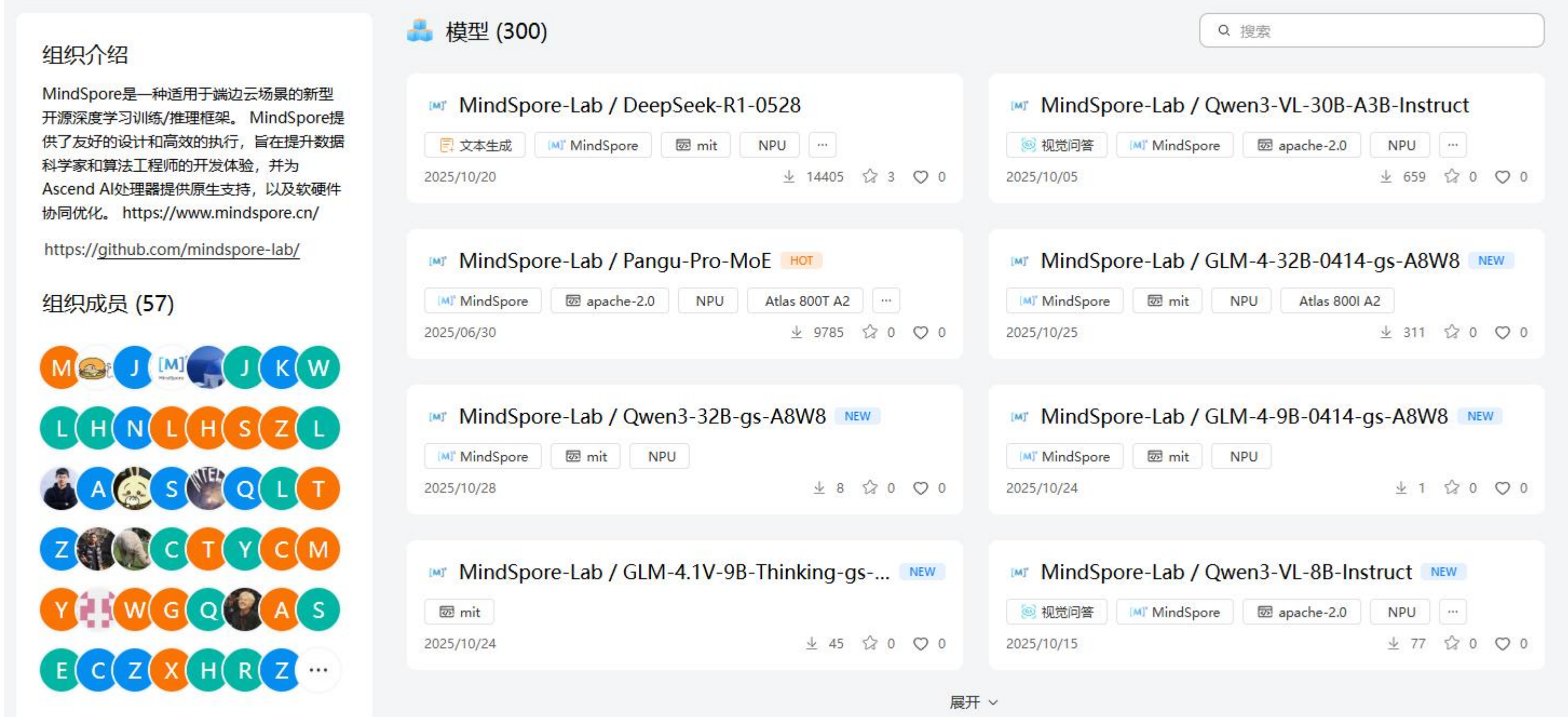
环境准备	量化校准	伪量化评测
1. 安装配套依赖包 2. Qwen3浮点权重 3. CEval校准集	• 构造量化配置项 • 一键校准 参考样例	• 构造量化配置项 • 一键评测 参考样例 伪量化评测主要用于简单测试量化权重的正确性（Demo特性）

已支持算法	简要介绍
OutlierSuppressionLite算法	由华为泰勒实验室与MindSpore团队联合开发，在SmoothQuant基础上为网络中的每个矩阵分别搜索最优超参 α 值
A8W4量化	由MindSpore团队开发的层间混合量化算法，激活使用动态per-token的8-bit量化，权重使用per-group的4-bit GPTQ量化
SmoothQuant算法	A8W8量化，通过smooth_scale将激活的量化难度转移至权重
GPTQ算法	对某个block内的所有参数逐个量化，弥补量化带来的精度损失
动态量化	激活或KVCache进行动态per-token量化
AWQ算法	通过离线网格搜索的方式实现权重低比特量化
RoundToNearest算法	朴素的后量化算法，其取整方式使用了四舍五入的方式

方式二：直接从魔乐社区或者Hugging Face社区下载

- Qwen3为例，8bit量化权重已经上传到：[MindSpore-Lab/Qwen3-32B-gs-A8W8 | 魔乐社区](#) 保存到本地。

• 可以网页下载也可以使用魔乐社区的API启动下载，参考：[下载接口 | 文档 | 魔乐社区](#)。假设本地保存路径为Qwen3-32B-gs-A8W8。



相关启动参数及调用命令

服务化参数	功能描述
--quantization/q {quantization_arg}	指定用于量化推理的方式，如果没有指定，首先检查模型配置文件中的quantization_config属性取值。如果为None，则不量化，并使用dtype来确定权重的数据类型。当前仅支持 “golden-stick” 和 “ascend” 。

- 金箍棒量化的权重：

```
vllm-mindspore serve Qwen3-32B-gs-A8W8
--port 8888
--tensor-parallel-size 4
--gpu-memory-utilization 0.75
--max-num-batched-tokens 4096
--block_size 32
--quantization golden-stick
```

- ModelSlim量化的权重：

```
vllm-mindspore serve Qwen3-32B-modelslim
--port 8888
--tensor-parallel-size 4
--gpu-memory-utilization 0.75
--max-num-batched-tokens 4096
--block_size 32
--quantization ascend
```

- 其他HF权重的权重：（待支持）

遵循vllm社区的使用规则

服务化示例

输入请求

```
> curl http://localhost:8888/v1/chat/completions -H "Content-Type: application/json" -d '{"model":"/data0/workspace/mindspore_ckpt/ckpt/qwen3/Qwen3-32B-osl", "messages":[{"role":"system","content":"You are a helpful assistant."},{ "role":"user","content":"2的十次方是多少？请直接给出答案："}], "max_tokens":512}'
```

指定量化模型并下发服务化请求

返回对话结果

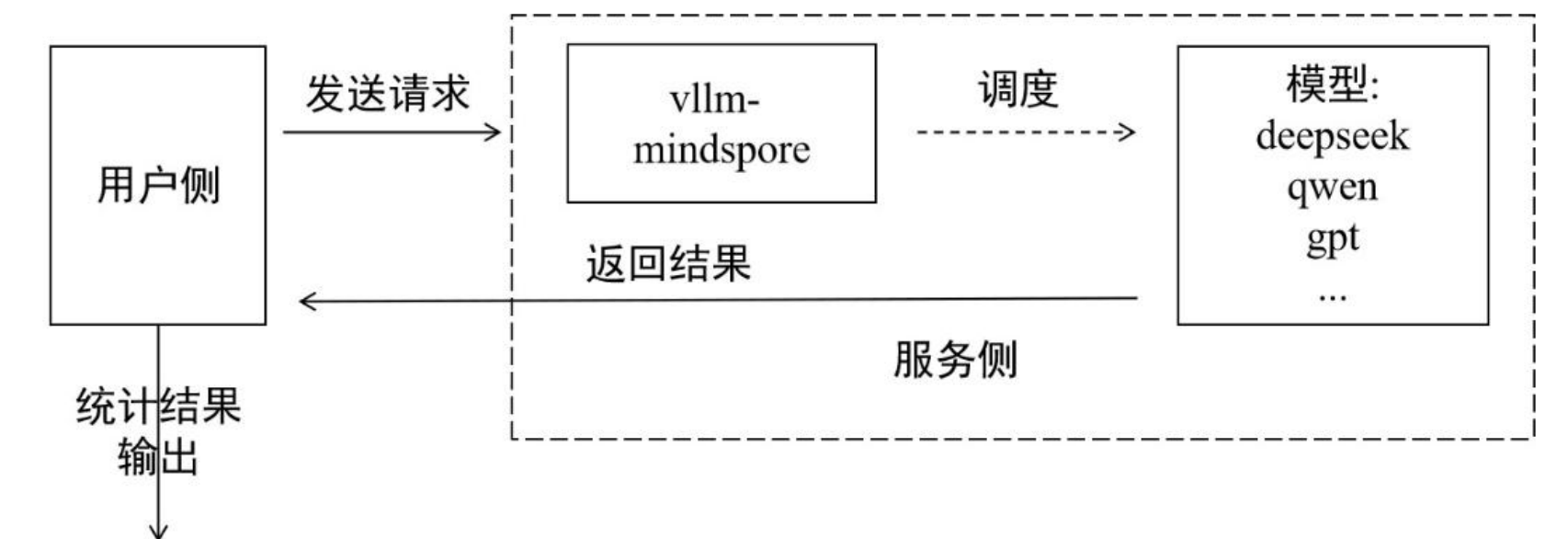
```
{ "id": "chatcmpl-4c86245f93b547a0ab1b3655074b43b2", "object": "chat.completion", "created": 1761633299, "model": "/data0/workspace/mindspore_ckpt/ckpt/qwen3/Qwen3-32B-osl", "choices": [{"index": 0, "message": {"role": "assistant", "reasoning_content": null, "content": "<think>\n\n嗯，用户问的是2的十次方是多少，让我直接给出答案。首先，我需要确认自己是否记得这个计算。2的十次方就是2乘以自己十次对吧？也就是2x2x2x...x2，共10个2相乘。\\n\\n让我先一步步计算看看。2的一次方是2，二次方是4，三次方是8，四次方是16，五次方是32，六次方是64，七次方是128，八次方是256，九次方是512，十次方应该是1024。对吧？\\n\\n不过，我是不是应该再检查一遍，确保没有计算错误。比如，2的五次方是32，没错。然后六次方是32x2=64，七次方是128，八次方256，九次方512，十次方就是1024。对的，没错。所以答案应该是1024。\\n\\n可能用户需要这个答案用于编程或者数学题，比如计算机科学中的内存单位，比如1KB是1024字节，所以这个数值在计算机领域很常见。用户可能是在做相关的题目，或者需要快速确认这个数值。直接给出答案的话，应该就是1024了。不过，有没有可能用户打错了次数？比如2的十次方是1024，但有时候可能会有混淆，比如2的十次方和2的十次方的平方之类的，但问题明确说是十次方，所以应该没问题。总之，答案应该是1024。\\n</think>\\n\\n2的十次方是1024。", "tool_calls": [], "logprobs": null, "finish_reason": "stop", "stop_reason": null}], "usage": {"prompt_tokens": 31, "total_tokens": 402, "completion_tokens": 371, "prompt_tokens_details": null}, "prompt_logprobs": null, "kv_transfer_params": null}
```

返回结果，计算结果1024，符合预期

服务化评测推荐[AISBench Benchmark](#)套件。AISBench Benchmark 是基于 OpenCompass 构建的模型评测工具，兼容 OpenCompass 的配置体系、数据集结构与模型后端实现，并在此基础上扩展了对服务化模型的支持能力。

当前，AISBench支持两大类推理任务的评测场景：

- **精度评测：**支持对服务化模型和本地模型在各类问答、推理基准数据集上的精度验证以及模型能力评估。
- **性能评测：**支持对服务化模型的延迟与吞吐率评估，并可进行压测场景下的极限性能测试。



评测前准备

Step1 安装AISBench评测环境

克隆仓库并通过源码安装：



```
git clone https://gitee.com/aisbench/benchmark.git
cd benchmark/
pip3 install -e ./ --use-pep517
```

Step2 数据集下载

支持开源官方数据集和自定义评测数据



```
cd ais_bench/datasets
mkdir ceval/
mkdir ceval/formal_ceval
cd ceval/formal_ceval
wget https://www.modelscope.cn/datasets/opencompass/ceval-exam/resolve/master/ceval-exam.zip
unzip ceval-exam.zip
rm ceval-exam.zip
```

Step3 启动vllm-mindspore服务

以GLM-4.5-Air为例



```
vllm-mindspore serve /home/checkpoint/GLM-4.5-Air
--trust_remote_code
--tensor_parallel_size=8
--max_num_seqs=192
--max_model_len=32768
--max_num_batched_tokens=16384
--block-size=32
--gpu-memory-utilization=0.93
```


精度评测

旨在评估模型在特定数据集上的预测准确率，如ceval、gsm8k、cmmlu等数据集。

Step1：更改接口配置

AISBench支持OpenAI的v1/chat/completions和v1/completions接口，在AISBench中分别对应不同的配置文件。以v1/completions接口为例，以下称general接口，需更改以下文件

[ais_bench/benchmark/configs/models/vllm_api/vllm_api_general_chat.py](#)配置：

```
from ais_bench.benchmark.models import VLLMCustomAPIChat

models = [
    dict(
        attr="service",
        type=VLLMCustomAPIChat,
        abbr='vllm-api-general-chat',

        # 指定模型序列化词表文件绝对路径，一般来说就是模型权重文件夹路径
        path="/home/checkpoint/GLM-4.5-Air",

        # 指定服务端已加载模型名称，依据实际VLLM推理服务拉取的模型名称配置（配置成空字符串会自动获取）
        model="GLM-4.5-Air",

        # 请求发送频率，每1/request_rate秒发送1个请求给服务端，小于0.1则一次性发送所有请求
        request_rate = 0,

        retry = 2,
        host_ip = "localhost",
        host_port = 8080,
        max_out_len = 512,
        batch_size=128,
        generation_kwargs = dict(
            temperature = 0.5,
            top_k = 10,
            top_p = 0.95,
            seed = None,
            repetition_penalty = 1.03,
        )
    )
]
```

Step2：命令行启动评测

AISBench参数	功能描述
--models	指定模型任务接口，如vllm_api_general，对应上一步更改的文件名。
--datasets	指定数据集任务。 如gsm8k_gen_0_shot_cot_str， gen表示生成式评测任务； shot表示提示的样本数为0； cot表示请求中包含思维链提示； str表示不含chat模板的输出；

启动评测：

```
ais_bench
--models vllm_api_general_chat
--datasets gsm8k_gen_0_shot_cot_str
```

返回结果示例：

```
The markdown format results is as below:
| dataset | version | metric | mode | vllm-api-general-chat |
|-----|-----|-----|-----|-----|
| gsm8k | 7cd45e | accuracy | gen | 90.45 |
```


性能评测

旨在评估服务模型的运行效率（吞吐、延迟），包括TTFT（首个Token返回的时延）、TPOT（每个Token的平均生成时延）、ITL（相邻Token间的平均间隔时延）等。

Step1：更改接口配置

配置与精度评测类似，以vllm_api_stream_chat任务为例，在

[ais_bench/benchmark/configs/models/vllm_api/vllm_api_stream_chat.py](#)更改如下配置：

```
from ais_bench.benchmark.models import VLLMCustomAPIChatStream

models = [
    dict(
        attr="service",
        type=VLLMCustomAPIChatStream,
        abbr='vllm-api-stream-chat',

        # 指定模型序列化词表文件绝对路径，一般来说就是模型权重文件夹路径
        path="/home/checkpoint/GLM-4.5-Air",

        # 指定服务端已加载模型名称，依据实际VLLM推理服务拉取的模型名称配置（配置成空字符串会自动获取）
        model="GLM-4.5-Air",

        # 请求发送频率，每1/request_rate秒发送1个请求给服务端，小于0.1则一次性发送所有请求
        request_rate = 0,

        retry = 2,
        host_ip = "localhost",          # 指定推理服务的IP
        host_port = 8080,              # 指定推理服务的端口
        max_out_len = 512,             # 推理服务输出的token的最大数量
        batch_size = 128,              # 请求发送的最大并发数
        generation_kwargs = dict(
            temperature = 0.5,
            top_k = 10,
            top_p = 0.95,
            seed = None,
            repetition_penalty = 1.03,
            ignore_eos = True,          # 推理服务输出忽略eos（输出长度一定会达到max_out_len）
        )
    ]
]
```

Step2：命令行启动评测

启动评测：

AISBench参数	功能描述
--mode	任务执行模式，如perf，默认为精度评测

```
ais_bench
--models vllm_api_stream_chat
--datasets gsm8k_gen_0_shot_cot_str_perf
--mode perf
```

返回结果示例：

指标	全称	说明
E2EL	End-to-End Latency	单个请求从发送到接收全部响应的总时延(ms)
TTFT	Time To First Token	首个 Token 返回的时延(ms)
TPOT	Time Per Output Token	输出阶段每个 Token 的平均生成时延（不含首个 Token）
ITL	Inter-token Latency	相邻 Token 间的平均间隔时延（不含首个 Token）

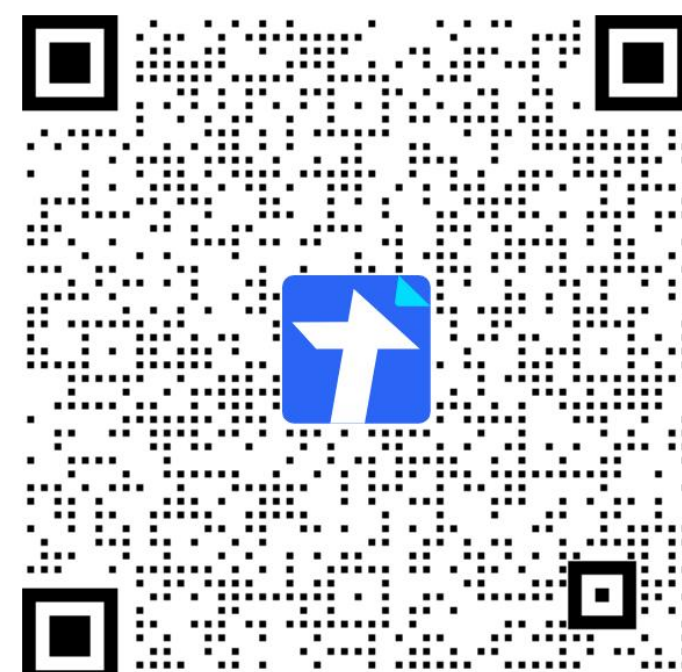
Performance Parameters	Stage	Average	Min	Max	Median	P75	P90	P99	N
E2EL	total	407854.2011 ms	319714.8954 ms	413187.3534 ms	408843.8573 ms	410466.8411 ms	411739.6943 ms	413172.1352 ms	1319
TTFT	total	834.2995 ms	170.3681 ms	1479.7493 ms	992.5189 ms	1095.9294 ms	1207.7726 ms	1344.7599 ms	1319
TPOT	total	81.404 ms	63.9037 ms	82.5834 ms	81.6021 ms	81.9135 ms	82.1851 ms	82.4729 ms	1319
ITL	total	81.3724 ms	0.0102 ms	674.7538 ms	81.8653 ms	85.4097 ms	87.7754 ms	91.7257 ms	1319
InputTokens	total	63.9826	27.0	190.0	60.0	75.0	94.0	129.0	1319
OutputTokens	total	5000.0	5000.0	5000.0	5000.0	5000.0	5000.0	5000.0	1319
OutputTokenThroughput	total	12.2701 token/s	12.101 token/s	15.6389 token/s	12.2296 token/s	12.2642 token/s	12.2931 token/s	15.6309 token/s	1319

SIG组织

昇思社区

昇腾社区

魔乐社区



课程交流群

MindSpore Transformers SIG群

MindSpore Transformers 内测文档

MindSpore Transformers 开源仓库

LLM Inference Serving SIG群

谢谢观看！

MindSpore Transformers & vLLM部署与评测

MindSpore Transformers 大模型系列课程